Frontier AI.
In Your Hands.



Kai Li
Applied AI



Florian Kowarsch
Applied AI

Mistral



Agenda

01 **Intro to Mistral & OCR**
Sovereign AI & Mistral OCR
8min

02 **Problem Introduction & Repo Setup**
Receipt Processing Introduced & Git Repository Setup
15min

03 **Reliable Workflows**
Stateful workflows with retries and recovery
12min

04 **Building Time**
Your time to get hands on and build an Mistral Workflow
20min

05 **Reliable Agents**
Evals & Observability
10min

$25 Credits

IMDA-DEV-DAY-26

A bit about Mistral…

- Operating in **25+ countries**
- Revenue **€400M+**
- €3B series D, valued at **€21B**
- Serving NVIDIA, Microsoft, ASML, & **100+ global leaders**

Arthur Mensch
Co-author of Chinchilla [Compute–optimal Scaling Laws]

Guillaume Lample
Co-author of Llama [Efficient, open-weight foundation models]

Timothée Lacroix
Co-author of Llama [Efficient, open-weight foundation models]

+ 1,300-strong team with a singular mission:

Empower customers to build, customize and own their AI, and ensure that its resulting value compounds with them.



Frontier open models



Customization and deployment



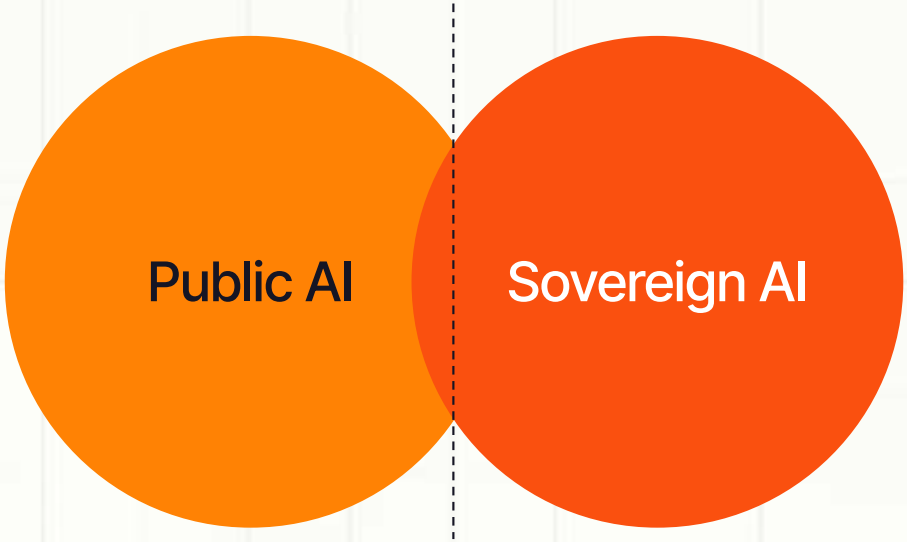
Enterprise agent platform



Sovereign AI Cloud



Enterprises are experiencing an AI divide...

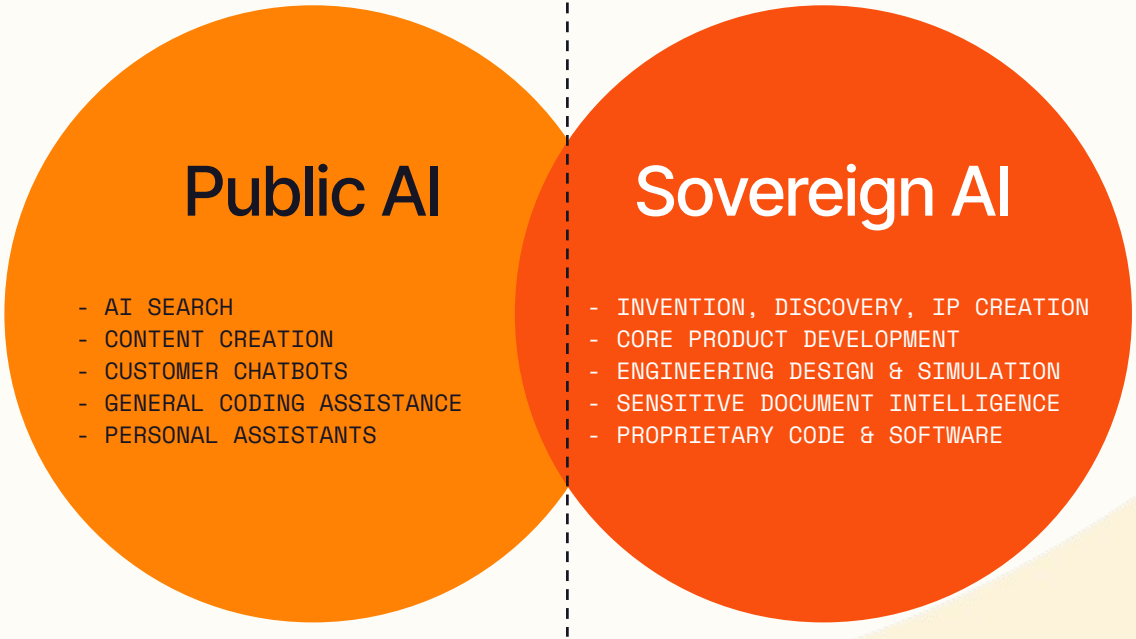


GENERAL PRODUCTIVITY

INTERNAL OPERATIONS WITH CUSTOMER DATA

IP-SENSITIVE WORKFLOWS

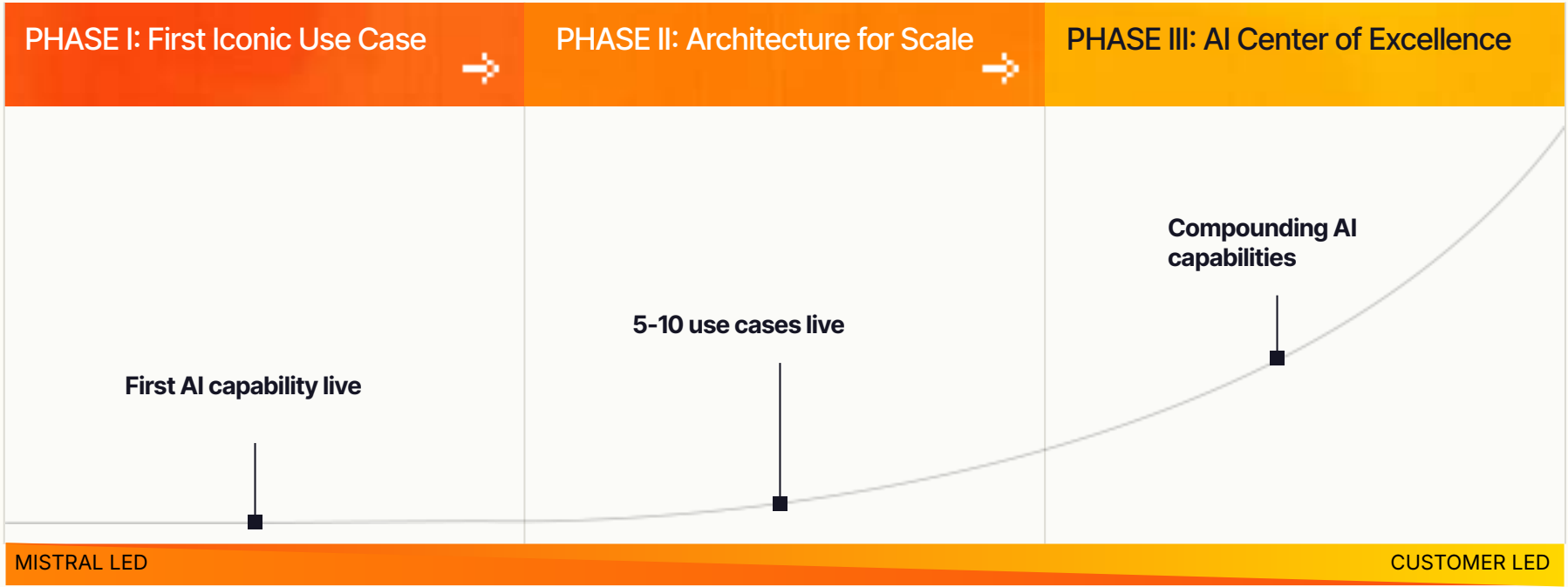
Enterprises are experiencing an AI divide...



Value compounds with the provider

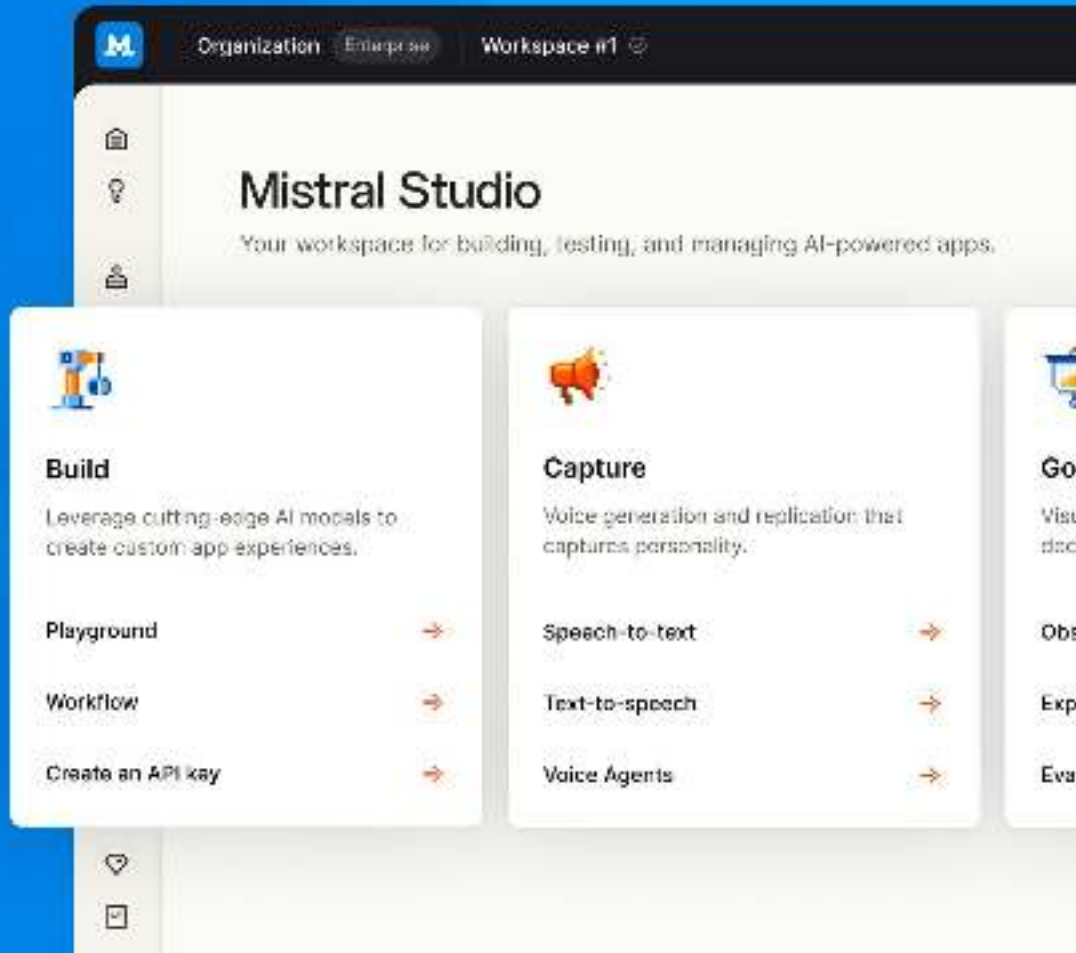
Value compounds to the customer

Journey to AI delivery excellence.



Studio

Build your own frontier



Introducing OCR 4.1

The market's best document extraction and understanding model

1k
Pages per minute per GPU

72%
Win rate over competitors in independent evaluations

170
Languages across 10 language groups

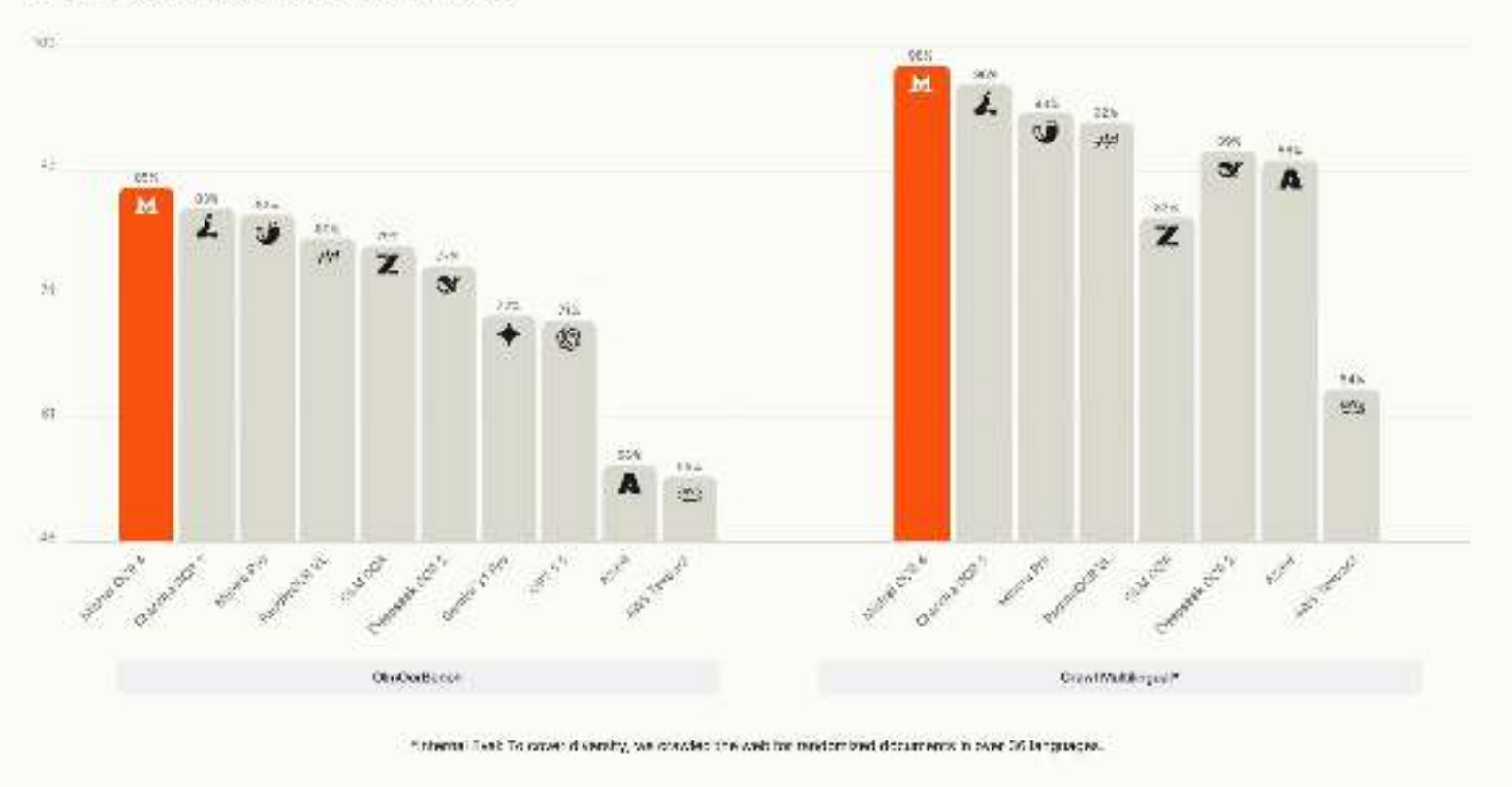
Key specs

- 66% increase in document throughput
- Bounding boxes on every text element - highlight sources in context and trace any extraction back to the paragraph
- Typed block classification - titles, footers, tables, equations, signatures, and more arrive pre-labeled
- Per-page and per-word confidence scores - enable routing only uncertain results to human review

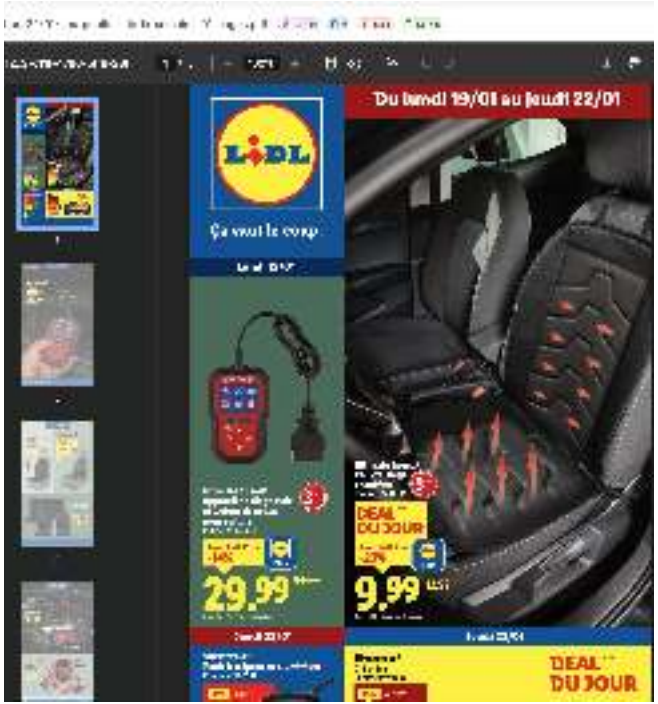
When to use

- Retrieval-Augmented Generation (RAG)
- Agentic workflows
- Structured data pipelines with confidence-gated human verification
- Enterprise search & knowledge bases

OCR model performance comparison



Extract structured output



OCR



Challenge Intro

Recipe Reconciliation



OCR

```
{
  "items": [
    ["BLACK PAPPER MEATBALL PAS", 1, "79,000"],
    ["TRUFFLE CREAM",             1, "77,000"],
    ["EARL GREY MILK TEA",        1, "59,000"]
  ],
  "subtotal": "215,000",
  "service": "12,900",
  "tax": "22,790",
  "total": "250,690"
}
```

Recipe Reconciliation



OCR

```
{
  "items": [
    ["BLACK PAPPER MEATBALL PAS", 1, "79,000"],
    ["TRUFFLE CREAM",             1, "77,000"],
    ["EARL GREY MILK TEA",        1, "59,000"]
  ],
  "subtotal": "215,000",
  "service": "12,900",
  "tax": "22,790",
  "total": "250,690"
}
```



Anomaly Detection

Git Repository

01 **Clone Repo**
`https://github.com/li-kai/mistral-demo`

02 **Create Mistral Account / Redeem Credits**
`https://console.mistral.ai`

03 **Create API Key**
`https://console.mistral.ai/api-keys`

04 **Configure in API Key**
Follow instructions in the repo

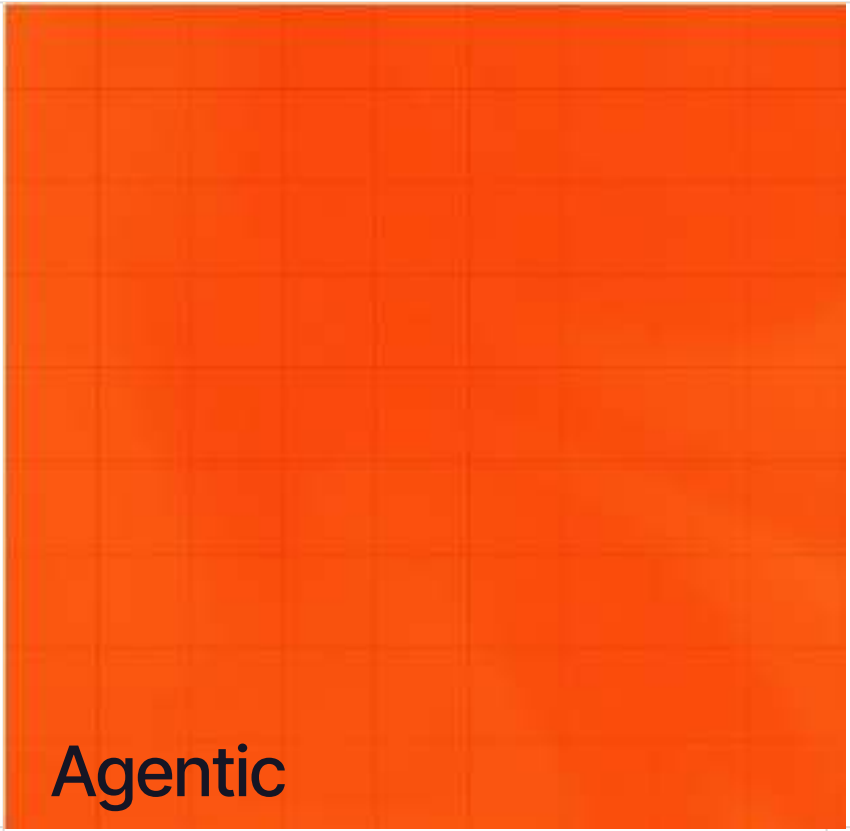
05 **Check README.md**
Setup env & start coding!

$25 Credits

`IMDA-DEV-DAY-26`

Two Types of Reliability





Two Types of Reliability



Rate Limits & Timeout



Faulty code & crashes



Retry & durable long execution

Operational

Agentic

Two Types of Reliability



Rate Limits & Timeout



Faulty code & crashes



Retry & durable long execution

Operational



Evaluation & Ground Truth



Observability



Confidence & Trust

Agentic

Orchestrate complex, agentic, and long-running processes with Workflows.

Durable execution

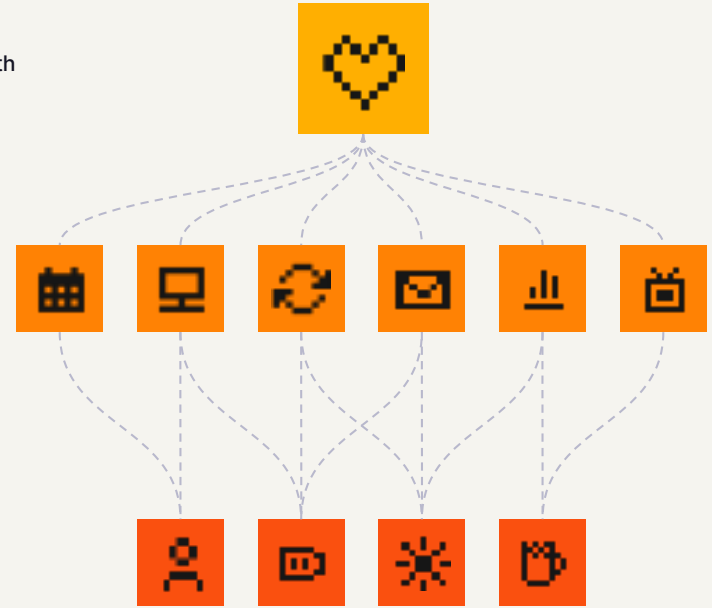
Stateful workflows with retries and recovery

Agentic coordination

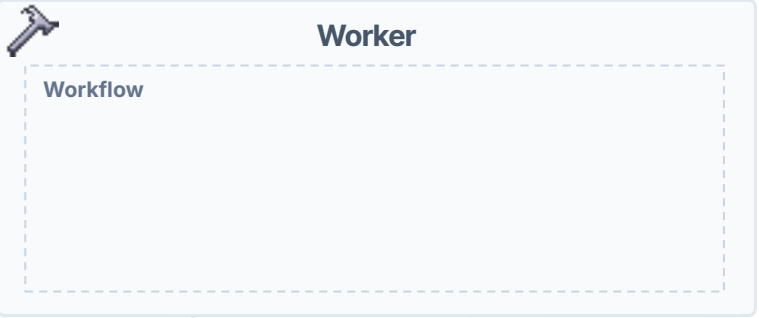
Orchestrate agents and tools

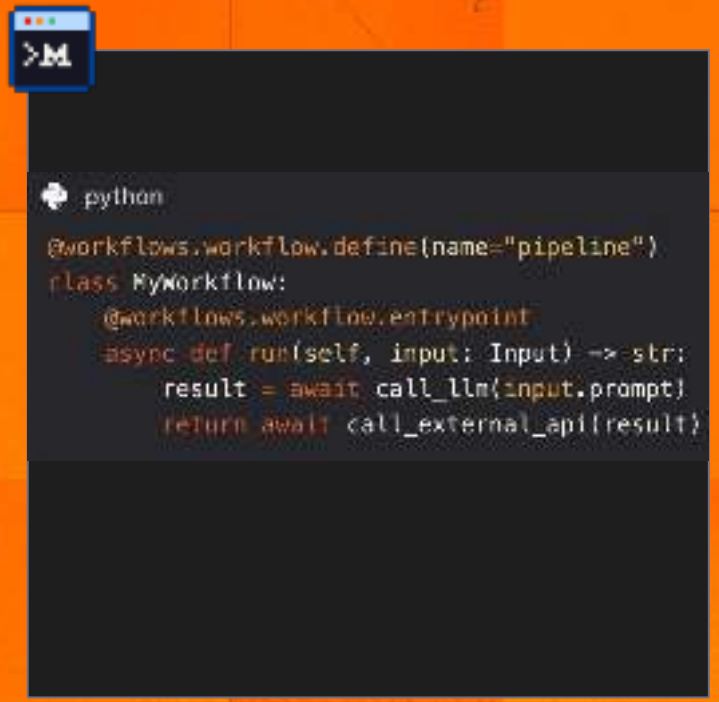
Enterprise integration

Embed directly into business systems



Mistral Workflows



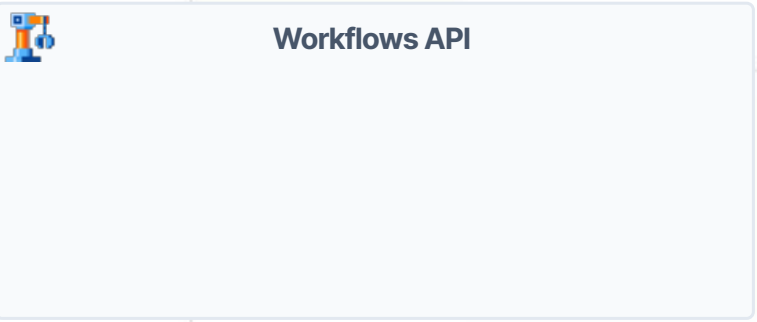


Mistral Workflows




```python
@workflows.activity(
    start_to_close_timeout=timedelta(minutes=5),
    retry_policy_max_attempts=3,
)
async def call_llm(prompt: str) -> str: ...
```

Mistral Workflows

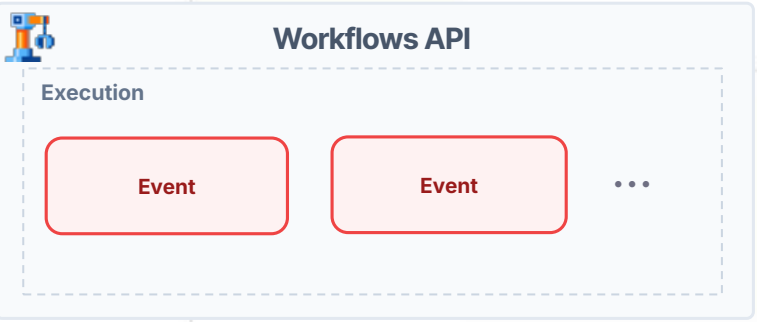





```python
client = Mistral(api_key="...")

execution = client.workflows.execute_workflow(
    workflow_identifier="support_agent",
    input={"ticket_id": "T-42"},
)
```

Mistral Workflows






```python
client = Mistral(api_key="...")

execution = client.workflows.execute_workflow(
    workflow_identifier="support_agent",
    input={"ticket_id": "T-42"},
)
```

Time To Build

01 **Clone & Setup Repo**
`https://github.com/li-kai/mistral-demo`

02 **Build your workflow & run it**

03 **Check out your workflow in Mistral Studio**
`https://console.mistral.ai/workflows`

Opt **Switch to `solution` branch for complete solution**



`github.com/li-kai/mistral-demo`

Branches and setup

Use phase-N-start for the exercise and phase-N-end to see the completed gap. Each end branch leads into the next start branch. Reinstall requirements when switching: the dependency list grows.

SETUP — from main to phase 1

```
git clone https://github.com/li-kai/mistral-demo.git
cd mistral-demo && git checkout phase-1-start
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env    # set MISTRAL_API_KEY
python check_key.py
# Next: phase-1-end → phase-2-start → phase-2-end → phase-3-start ...
```

FULL REFERENCE

`git checkout solution` — main is only the starter; there is no solution tag

OPERATIONAL

Reconciliation rules R1–R5

Your prompt must apply R1 to R5 and answer with one of: approve, partial, reject, escalate.

RULE	CHECK
R1	Claimed total matches the receipt total (about 1%)
R2	Tax counted once
R3	Discount applied (post-discount price)
R4	Every claimed item is on the receipt, same amount. If not: escalate
R5	Change and alcohol are not claimed

OPERATIONAL

Single-prompt reconciliation

One chat call sends the claim JSON and committed OCR text to mistral-medium-latest. The raw free-text reply is not a typed decision: its wording and decision may vary. Your job is to build the prompt and make the call.

naive_chat.py — two separate TODOs (abridged)

```
CHAT_MODEL = "mistral-medium-latest"
def build_prompt(claim, receipt_text) -> str:
    # TODO: one prompt — expense auditor, the claim as JSON,
    #       the receipt text, one of DECISION_LABELS
    raise NotImplementedError("write the prompt")
def main() -> None:
    # TODO: call chat.complete(...) and print the raw reply
    raise NotImplementedError("call the chat model")
```

DONE WHEN

`python naive_chat.py --claim claims/claim_b.json → compare 3 raw replies`

OPERATIONAL

Output varies between runs

Run claim_a (clean) once. Run claim_b three times.

Same claim: different decision? Different format?

Can a program read the answer?

DEMO

```
python naive_chat.py --claim claims/claim_b.json   # x3
```

OPERATIONAL

Structured extraction

One mistral-ocr-latest call annotates the receipt image with a Pydantic Receipt schema. Validation makes missing or mistyped required fields visible. Add payment_method and change; do not confuse cash tendered with the bill total.

extract.py — Receipt fields (descriptions omitted)

```
class Receipt(BaseModel):
    merchant: str | None
    items: list[Item]
    subtotal: float | None
    tax: float | None
    discounts: list[Discount]
    total_paid: float | None
   # TODO: payment_method: str | None (cash tendered OR change printed)
    # TODO: add change: float | None  (the cash returned to the customer)
```

DONE WHEN

`python extract.py --receipt receipts/r0001.jpg → cash + change: 6500`

OPERATIONAL

Rule-based reconciliation

Given a fixed extraction, code applies R1 totals, R2 tax, R3 discounts, R4 item-mismatch escalation, and R5 change/alcohol exclusions. Live OCR can vary; the rules themselves are deterministic. Implement R2 and R4.

reconcile.py — R2 and R4

```
def check_escalation(claim, receipt) -> list[str]:        # R4
    # TODO: missing total, no line items, or an item-level mismatch
    #       beyond tolerance -> one reason per problem, then escalate
    return []
def check_tax_once(claim, receipt) -> tuple[list[str], float]:   # R2
    # TODO: compare tax_claimed with the receipt tax, using close()
    return [], 0.0
```

DONE WHEN

`python reconcile.py --all` → target: 3/5 before, 5/5 after the fix

OPERATIONAL

Rule-based results

Run it twice. The decisions do not change.

Before the fix: 3/5 (claim_b has no R2, claim_d has no R4).

Claims f to i are skipped: no simple rule catches "10 cats".

DEMO

```
python reconcile.py --all   # 5/5 PASS
```

OPERATIONAL

Durable workflow

The Mistral-hosted orchestrator schedules local extract and reconcile activities. Add a 2-minute start-to-close timeout and 3 attempts to extract_activity so a seeded first-attempt OCR failure can recover.

workflow.py — retry policy

```
# TODO: add a retry policy, like reconcile_activity:
#       start_to_close_timeout=timedelta(minutes=2), retry_policy_max_attempts=3
@workflows.activity(retry_policy_max_attempts=1)
async def extract_activity(receipt_path: str) -> dict:
    """OCR the receipt image and extract typed fields in one call."""
    attempt = temporal_activity.info().attempt
    # seeded demo: fails once on receipts/r0003.jpg, then attempt=2 succeeds
```

DONE WHEN

`python worker.py & python run_workflow.py --claim claims/claim_c.json`

OPERATIONAL

Workflow retry demo

Worker log: attempt=1 fails, attempt=2 succeeds.

Execution ends COMPLETED. With one attempt it ends FAILED.

Studio does not mark the failed attempt. The worker log does.

DEMO

```
python worker.py &
python run_workflow.py --claim claims/claim_c.json
```

Dataset & Task







python

```python
import csv

dataset = list(csv.DictReader(open("golden_set.csv")))
# [{"question": "...", "expected": "..."}, ...]  - plain dicts, your keys

async def task(ctx: TaskContext) -> str:              # your pipeline under test
    r = await client.chat.complete_async(
        model=ctx.system.params["model"],
        messages=[
            {"role": "system", "content": ctx.system.params["system_prompt"]},
            {"role": "user",   "content": ctx.input_record["question"]},
        ],
    )
    return str(r.choices[0].message.content)
```

AGENTIC

Anomaly judge (R6)

The judge reads raw OCR text and returns a validated AnomalyVerdict for R6. Flag non-food items, groups over four, or implausible per-person prices; leave tax, discounts, alcohol and change to rules. Write INSTRUCTIONS, then evaluate it.

anomaly.py — abbreviated judge skeleton

```
ANOMALY_MODEL = "mistral-medium-3-5"
REASONING_EFFORT = "high"
# TODO: flag non-food items, >4 diners, ~10x prices per person
# exclude alcohol, tax, discounts, change and unclear OCR
INSTRUCTIONS = """You audit expense claims..."""
class AnomalyVerdict(BaseModel):
    reasons: list[str]
    is_anomalous: bool
# detect_anomaly returns the typed verdict consumed by R6
```

TRY IT

`python evaluate.py → target: 5/5 anomalies found; 0/4 clean flagged`

AGENTIC

Judge verdict and final decision

The workflow obtains OCR fields and raw text, asks the judge for an R6 signal, then applies explicit decision precedence. A mismatched claim is escalated even if the judge also flags an anomaly.

workflow.py + reconcile.py — abridged excerpts

```
extraction = await extract_activity(claim["receipt"])
anomaly = await anomaly_activity(claim, extraction)
result = await reconcile_activity(claim, extraction, anomaly)
escalation = check_escalation(claim, receipt)
anomalies = check_anomaly(anomaly)
# R4: escalate with escalation + anomalies; else R6: reject
# otherwise R1/R2/R3/R5 calculate reimbursable_amount
```

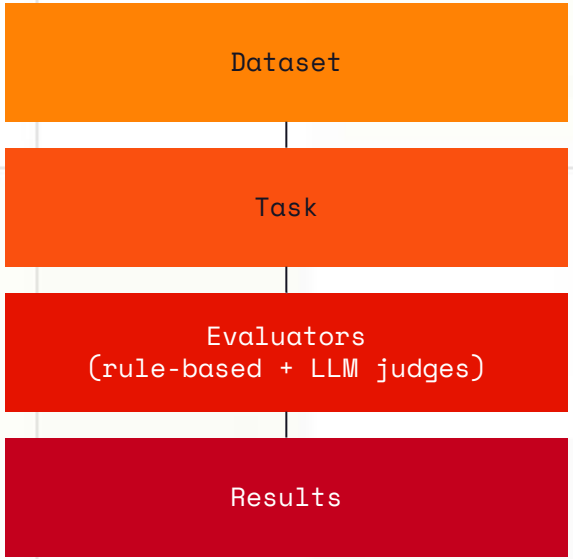
EXAMPLES

claim_d: escalate + R6 reason | claim_g: reject | claim_e: partial

How do you know it still works?



How do you know it still works?



Run Eval







python

Copy

```python
def accuracy(ctx: ScorerContext) -> int:
    return int(str(ctx.output).strip().lower() == ctx.input_record["expected"].lower())

for name, params in {
    "prompt-v1": {"model": "mistral-small-latest", "system_prompt": V1},
    "prompt-v2": {"model": "mistral-small-latest", "system_prompt": V2},
}.items():
    run = await client.evaluation.run(
        evaluation=Evaluation(name="Prompt comparison"),
        system=System(name=name, params=params),     # config recorded on the run
        dataset=dataset,
        task=task,
        evaluators=[Evaluator(name="accuracy", scorer=accuracy, goal=Goal.gte(0.8))],
    )
    run.show(level="run")   # stats in terminal; runs & systems compared in Studio
```

AGENTIC

Judge evaluation

Use the nine committed OCR texts to compare AnomalyVerdict with expected_anomaly, without introducing another OCR call. The local evaluation reports precision, recall, F1 and a confusion matrix. Then test the complete pipeline on live receipts.

EVALUATE THE SIGNAL, THEN THE DECISION

```
python evaluate.py
# target: 5 labelled anomalies found; 0 of 4 clean claims flagged
python evaluate.py --reasoning-effort none
python evaluate.py --model mistral-large-4
python reconcile.py --all
# target: 9/9 PASS; live OCR and model calls may vary
```

READ THE METRICS

Precision: false alarms | Recall: missed anomalies | F1: balance

AGENTIC

Judge variability

A model judge is noisy: run each setting more than once.

The model judges. The decision stays in rules you can test.

DEMO

```
python reconcile.py --all   # 9/9 PASS, most of the time
```

Thank you



Kai Li
Applied AI



Florian Kowarsch
Applied AI

Mistral

We are hiring!



34 open roles across APAC

https://mistral.ai/careers